Lab 5.1: Creating Controls Dynamically

In this lab, you will create a bitmap button control dynamically.

To see a demonstration of the lab solution, click this icon.
(CD-ROM plays the demonstration, "Lab 5.1 Demonstration.")

Estimated time to complete this lab: **30 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch05\Lab5.1\Ex01.

The solution code for this lab is located in the folder \Labs\Ch05\Lab5.1\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Create the resources to support a bitmap button control.
- Create a bitmap button control dynamically.
- Handle the BN_CLICKED notification message sent by the bitmap button control.

Prerequisites

Before working on this lab, you should be familiar with the following:

- Combo boxes
- Dialog bars
- Rebars

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter:

Exercise 1: Creating a Bitmap Button Control Dynamically

In this exercise, you will create a bitmap button control dynamically and handle the BN_CLICKED notification message sent by the control.

158 Exercise 1: Creating a Bitmap Button Control Dynamically

The code that forms the starting point for this exercise is in \Labs\Ch05\Lab5.1\Ex01.

In this exercise, you will create a bitmap button control dynamically and handle the BN_CLICKED notification message sent by the control. You will create a button that will allow the user to search for a string in a document.

▶ Create an ID for the bitmap button control

☐ 1. Use ResourceView to open the String table folder.

☐ 2. Open the properties window by double-clicking **String table**.

☐ 3. Display the properties for the blank line at the end of the String table resource.

Set **ID** to ID_BITMAPBUTTON and **Caption** to Search string\nSearch.

☐ 4. Save the file BitmapButton.rc.

▶ Create the dialog bar resources

☐ 1. Use ResourceView to insert a new dialog resource.

☐ 2. Expand the Dialog icon and click **IDD_DIALOG1**.

☐ 3. Delete both the **OK** and **Cancel** buttons.

☐ 4. Open the **Dialog Properties** dialog box.

☐ 5. On the **General** tab, clear the caption.

☐ 6. On the **Styles** tab, set **Style** to Child and **Border** to None.

☐ 7. Resize the dialog box to 266 by 18 pixels.

☐ 8. Draw a combo box from the position 2,1 sized to 146 by 12 pixels.

☐ 9. On the **Styles** tab, set **Type** to Dropdown and **Owner draw** to No. Click **Vertical Scroll**.

☐ 10. Save BitmapButton.rc.

▶ Add a protected variable to CMainFrame class

In this procedure, you will create a **CReBar** object.

- Right-click **CMainFrame** in ClassView and add a protected variable as follows:

```
CReBar m_wndReBar;
```

▶ Create a bitmap button

☐ 1. Open the file MainFrm.h and create a **CBitmapButton** object in the public attributes section as follows:

```
CBitmapButton m_ctlBitmapButton;
```

☐ 2. Open the file MainFrm.cpp and declare a **CDialogBar** object after the declaration of the status line indicators as follows:

```
CDialogBar m_wndDialogBar;
```

Notes

☐ 3. In the **OnCreate** function, call the **CDialogBar::Create** function using the data member m_wndDialogBar. Place the code after the call to **CToolBar::CreateEx** as follows:

```
if (!m_wndDialogBar.Create(this, IDD_DIALOG1, CBRS_ALIGN_TOP,
AFX_IDW_DIALOGBAR))
{
    TRACE0("Failed to create dialogbar\n");
    return -1;      // fail to create
}
```

☐ 4. Create the bitmap button control as follows:

```
m_ctlBitmapButton.Create("Search",WS_CHILD | WS_VISIBLE | BS_OWNERDRAW
,CRect(230,3,262,32),&m_wndDialogBar,ID_BITMAPBUTTON);
```

☐ 5. Call the **LoadBitmaps** member function to load the bitmap resources after the bitmap button is constructed as follows:

```
m_ctlBitmapButton.LoadBitmaps( IDB_BITMAP1,IDB_BITMAP2,0,0);
```

☐ 6. Create a **CReBar** object and add controls to it using the **AddBar** function as follows:

```
if (!m_wndReBar.Create(this) ||
    !m_wndReBar.AddBar(&m_wndToolBar) ||
    !m_wndReBar.AddBar(&m_wndDialogBar))
{
    TRACE0("Failed to create rebar\n");
    return -1;      // fail to create
}
```

Notes

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
int CMainFrame::OnCreate(LPCREATESTRUCT lpCreateStruct)
{
    if (CFrameWnd::OnCreate(lpCreateStruct) == -1)
        return -1;
        if (!m_wndToolBar.CreateEx(this) ||
!m_wndToolBar.LoadToolBar(IDR_MAINFRAME))
    {
        TRACE0("Failed to create toolbar\n");
        return -1;      // fail to create
    }

    if (!m_wndDialogBar.Create(this, IDD_DIALOG1, CBRS_ALIGN_TOP,
AFX_IDW_DIALOGBAR))
    {
        TRACE0("Failed to create dialogbar\n");
        return -1;      // fail to create
    }
    m_ctlBitmapButton.Create("Search",WS_CHILD | WS_VISIBLE | BS_OWNERDRAW
,CRect(230,3,262,32),&m_wndDialogBar,ID_BITMAPBUTTON);
    m_ctlBitmapButton.LoadBitmaps( IDB_BITMAP1,IDB_BITMAP2,0,0);

    if (!m_wndReBar.Create(this) ||
        !m_wndReBar.AddBar(&m_wndToolBar) ||
        !m_wndReBar.AddBar(&m_wndDialogBar))
    {
        TRACE0("Failed to create rebar\n");
        return -1;      // fail to create
    }

    if (!m_wndStatusBar.Create(this) ||
        !m_wndStatusBar.SetIndicators(indicators,
          sizeof(indicators)/sizeof(UINT)))
    {
        TRACE0("Failed to create status bar\n");
        return -1;      // fail to create
    }
    m_wndToolBar.SetBarStyle(m_wndToolBar.GetBarStyle() |
    CBRS_TOOLTIPS | CBRS_FLYBY);

    return 0;
}
```

Notes

▶ **Create message map entries for the combo box**

☐ 1. Run the dialog editor on the IDD_DIALOG1 dialog resource and select the combo box.

☐ 2. Invoke ClassWizard.

The **Adding a class** dialog box will automatically appear. Click **Cancel** to dismiss this dialog box.

☐ 3. In the **Message Maps** tab, set **Class name** to CBitmapButtonView, set **Object ID** to IDC_COMBO1, and set **Message** to CBN_KILLFOCUS.

☐ 4. Click **Add Function** and accept the default name of **OnKillfocusCombo1**.

▶ **Create message map entries for the Windows notification message BN_CLICKED**

☐ 1. Open the file BitmapButtonView.h and add the following line after the //{{AFX_MSG message block:

```
afx_msg void OnBnClicked()
```

☐ 2. Insert a message map entry for the **OnBnClicked** function in the **CBitmapButtonView** implementation file. Place the entry between the end of the code section created by ClassWizard and the end of the message map as follows:

```
    //{{AFX_MSG_MAP(CBitmapButtonView)
    ON_CBN_KILLFOCUS(IDC_COMBO1, OnKillfocusCombo1)
    //}}AFX_MSG_MAP
    ON_BN_CLICKED(ID_BITMAPBUTTON,OnBnClicked)
END_MESSAGE_MAP()
```

For more information about handling notification messages from a bitmap button, see Processing Notification Messages in this chapter.

▶ **Add a protected variable to the CBitmapButtonView class**

- Right-click **CBitmapButtonView** in ClassView and add a protected variable as follows:

```
CString m_text;
```

▶ **Add an extern declaration for CDialogBar**

- Because m_wndDialogBar is declared in MainFrm.cpp, add the following declaration to the CBitmapButtonView.h before the definition of **CBitmapButtonView**:

```
extern CDialogBar m_wndDialogBar;
```

▶ **Write the code for the OnKillfocusCombo1 function**

In this handler, you will add the string typed by the user to the combo box if it is a new string.

- Write the code for the **OnKillfocusCombo1** function as follows:

```
CComboBox *ptr;
ptr=(CComboBox*)m_wndDialogBar.GetDlgItem(IDC_COMBO1);
m_wndDialogBar.GetDlgItemText(IDC_COMBO1,m_text);

if(ptr)
{
    if(ptr->FindStringExact(0,m_text)==CB_ERR&& !m_text.IsEmpty())
        ptr->AddString(m_text);
}
```

Notes

▶ Write the code for the OnBnClicked function

In this handler, you will search for the entered string in a document.

- Write the code for the **OnBnClicked** function as follows:

```
void CBitmapButtonView::OnBnClicked()
{
if(!m_text.IsEmpty())
    OnFindNext(m_text,TRUE,TRUE);
else
::MessageBox(0,"Enter search string","Search",MB_OK);
}
```

▶ Override OnTextNotFound function to change the default implementation

In the default implementation, the **OnFindNext** function calls the **OnTextNotFound** function if the string is not found.

☐ 1. Add the **OnTextNotFound** function to the **CBitmapButtonView** class as a protected function. It should be added after the //{{AFX_VIRTUAL block:

```
virtual void OnTextNotFound(LPCTSTR lpszFind);
```

☐ 2. Write the code for the **OnTextNotFound** function as follows:

```
::MessageBox(0,"End of file reached","End of file",MB_OK);
```

▶ Build and test the application

- Build your application. Open a file or enter text in EditView. Enter the string to be searched and click **Search**. If the string is found, it should be highlighted. If the string is not found, a message box indicating the end of file should appear.

The solution code for this exercise is located in the folder Labs\Ch05\Lab5.1\Ex01\Solution.

Notes